

UNIVERSIDAD TECNOLÓGICA NACIONAL TECNICATURA UNIVERSITARIA EN PROGRAMACIÓN A DISTANCIA

PROGRAMACIÓN 2 TRABAJO PRÁCTICO INTEGRADOR – FOOD STORE

Integrantes:

Alejandro Daniel Maure Legajo: 18289

Lautaro Ezequiel Mansilla Legajo: 34293

ÍNDICE

Introducción y cumplimiento de la consigna	2
Marco teórico	4
Decisiones técnicas y arquitectura	7
Dificultades y soluciones	15
Historias de usuario y código	17
Bibliografía y webgrafía	18

1. INTRODUCCIÓN Y CUMPLIMIENTO DE LA CONSIGNA

Food Store es una aplicación de consola desarrollada en Java 21 que permite gestionar categorías, productos, usuarios y pedidos de un negocio de comidas.

El sistema implementa operaciones CRUD completas mediante menús interactivos, sin login, con persistencia en memoria mediante colecciones Java y arquitectura por capas (entidades, repositorios, servicios y vista de consola).

1.1 Alcance implementado

Requisito de la consigna	Estado
Aplicación de consola con menú principal	Cumplido
Java 21	Cumplido
POO: herencia, encapsulamiento, polimorfismo	Cumplido
UML: Base, entidades, relaciones 1:N y N:1	Cumplido
Enums Rol, Estado, FormaPago	Cumplido
Interfaz Calculable en Pedido	Cumplido
Colecciones en memoria (sin base de datos)	Cumplido
Excepciones propias + try-catch	Cumplido
Soft delete (campo eliminado = true)	Cumplido
4 épicas / 16 historias de usuario	Cumplido
Separación Main / Service / Entities / UI	Cumplido
Sin Login, sin JDBC	Cumplido

1.2 Reglas de negocio implementadas

- Precio y stock de producto ≥ 0 (excepción si no se cumple).
- No se crea pedido sin usuario activo.
- Cantidad de detalle > 0 .
- Mail de usuario único (validación recorriendo la colección).
- Nombre de categoría único entre categorías activas.
- Soft delete: no se eliminan objetos físicamente del Map.
- Categoría con productos activos no puede darse de baja.
- Producto referenciado en pedidos no se borra del Map (solo marca eliminado).
- Pedidos históricos de usuario eliminado siguen consultables.
- Creación de pedido transaccional: si falla un detalle, no se persiste.
- Uso obligatorio de `addDetallePedido()` y `calcularTotal()` vía `Calculable`.

2. MARCO TEÓRICO

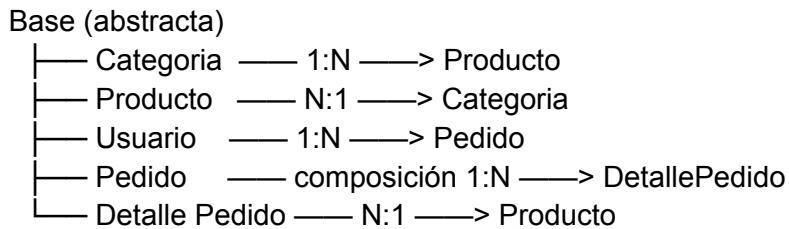
2.1 Java como lenguaje de implementación

Java es un lenguaje orientado a objetos, compilado a bytecode y ejecutado en la JVM. Se eligió Java 21 por ser la versión indicada en la consigna. Entre sus características aplicadas en este proyecto:

- Tipado estático: errores detectables en compilación.
- Encapsulamiento: atributos privados con getters/setters.
- Herencia: clase abstracta Base compartida por todas las entidades.
- Polimorfismo: sobrescritura de toString(), equals(), hashCode(); interfaz Calculable implementada por Pedido.
- Manejo de excepciones checked: ValidacionException, EntidadNoEncontradaException, StockInvalidoException propagadas y capturadas en capa UI.
- API de fecha/hora: LocalDateTime (auditoría), LocalDate (fecha de pedido).
- Colecciones del java.util: HashMap, ArrayList.
- Scanner para entrada estándar en consola.

2.2 Programación Orientada a Objetos (POO)

El diseño sigue el UML provisto:



La clase Base centraliza id, eliminado y createdAt. Las entidades de dominio contienen comportamiento propio (por ejemplo, Pedido.addDetallePedido), no son simples DTOs anémicos.

2.3 Colecciones y persistencia en memoria

En lugar de una base de datos relacional, cada repositorio mantiene un `HashMap<Long, Entidad>` como almacén. Ventajas en el contexto académico:

- Simplicidad y foco en POO y capas.
- Acceso $O(1)$ por id.
- Filtrado de activos recorriendo `values()` y verificando `!eliminado`.

Limitación consciente: los datos se pierden al cerrar el programa. La consigna lo define así.

2.4 Bases de datos relacionales (contexto teórico, no implementado)

Aunque el título del TP menciona JDBC, la entrega utiliza memoria. En un escenario productivo, las entidades se mapearían a tablas (categorías, productos, usuarios, pedidos, detalle_pedido) con claves foráneas. JDBC permitiría conectar Java con MySQL, PostgreSQL u otro motor SQL mediante `DriverManager`, `PreparedStatement` y `ResultSet`. Este proyecto deja preparado el modelo de dominio para una futura capa de persistencia sin cambiar la lógica de negocio en servicios.

2.5 Interfaces y enums

- Calculable: contrato polimórfico con método `calcularTotal()`. Pedido lo implementa sumando subtotales de detalles.
- Rol, Estado, FormaPago: tipos cerrados que restringen valores válidos y evitan strings mágicos en el dominio.

2.6 Arquitectura por capas

Capa UI (ui/)	-> Scanner, menús, mensajes al usuario.
Capa Service (service/)	-> Casos de uso, validaciones, orquestación.
Capa Repository	-> Acceso a colecciones, generación de IDs.
Capa Entities	-> Modelo de dominio UML.
Capa Exception	-> Errores de negocio tipados.

Principio: Main y menús no contienen reglas de negocio complejas; delegan en services. Los services no leen de Scanner; reciben datos ya parseados.

3. DECISIONES TÉCNICAS Y ARQUITECTURA

3.1 Estructura de paquetes

tupad-tpi-programacion2/

└─ src/tpi/

├─ Main.java

├─ AppContext.java

├─ entities/

├─ enums/

├─ interfaces/

├─ exception/

├─ repository/

├─ service/

└─ ui/

Punto de entrada

Composición de dependencias (DI manual)

Modelo UML

Rol, Estado, FormaPago

Calculable

Excepciones propias

Persistencia en memoria

Lógica de aplicación

Menús y utilidades de consola

3.2 ApplicationContext como contenedor liviano

En lugar de un framework de inyección de dependencias, ApplicationContext instancia un repositorio y un servicio por entidad en el constructor. Los menús obtienen solo el service que necesitan vía getters. Esto mantiene el proyecto acorde a Programación 2 sin dependencias externas.

Cadena de dependencias:

CategoriaRepository -> CategoriaService

ProductoRepository + CategoriaRepository -> ProductoService

UsuarioRepository -> UsuarioService

PedidoRepository + UsuarioRepository + ProductoRepository -> PedidoService

3.3 Patrón Repository

Cada repositorio encapsula el Map y la secuencia de IDs. Los services no conocen la estructura interna del almacén; solo invocan guardar, buscarPorId, listarActivos, etc.

3.4 Soft delete

Todas las bajas marcan eliminado = true (borrado lógico). Los listados filtran registros activos.

Los objetos permanecen en memoria para integridad referencial (pedidos antiguos siguen apuntando a productos o usuarios “eliminados”).

3.5 Flujo de creación de pedido (decisión crítica)

1. UI recolecta usuario, estado, forma de pago y lista de DetalleSolicitud.
2. PedidoService valida usuario activo y que haya al menos un detalle.
3. Se crea Pedido en memoria (aún NO en repositorio global).
4. Por cada detalle: addDetallePedido() valida stock y disponibilidad.
5. calcularTotal() vía interfaz Calculable.
6. descontarStockProductos() actualiza stock en productos.
7. Solo si todo OK: pedidoRepository.guardar() y usuario.agregarPedido().

Si cualquier paso lanza excepción, el pedido nunca se guarda -> sin datos inconsistentes (requisito HU-PED-02).

3.6 ConsolaUtil

Clase utilitaria estática para lectura robusta: reintenta ante `NumberFormatException` y valida rangos. Evita duplicar bucles while en cada menú y previene cierre inesperado del programa por entrada inválida.

3.8 Capturas de pantalla

Comprobar version de java carpeta out con el programa compilado

```
1 alejandro@alejandro-... +   
  
→ tpi_programacion_2 java -version  
openjdk version "21.0.11" 2026-04-21  
OpenJDK Runtime Environment (build 21.0.11+10-1-26.04.2-Ubuntu)  
OpenJDK 64-Bit Server VM (build 21.0.11+10-1-26.04.2-Ubuntu, mixed mode, sharing)  
→ tpi_programacion_2 ls  
out src  
tpi_programacion_2 █
```

Menú principal al iniciar la aplicación.

```
→ tpi_programacion_2 java -cp out tpi.Main  
  
=== SISTEMA DE PEDIDOS (FOOD STORE) ===  
1. Categorías  
2. Productos  
3. Usuarios  
4. Pedidos  
0. Salir  
Seleccione: █
```

Listado de categorías después de crear al menos una.

```
--- CATEGORÍAS ---
1. Listar
2. Crear
3. Editar
4. Eliminar
0. Volver
Seleccione: 2
Nombre: Hamburguesas
Descripción: Listado de hamburguesas
Categoría creada con id: 1

--- CATEGORÍAS ---
1. Listar
2. Crear
3. Editar
4. Eliminar
0. Volver
Seleccione: 1

Categorías activas:
[1] Hamburguesas | Listado de hamburguesas

--- CATEGORÍAS ---
1. Listar
2. Crear
3. Editar
4. Eliminar
0. Volver
Seleccione:
```

Alta de producto con categoría asociada.

```
=== SISTEMA DE PEDIDOS (FOOD STORE) ===
1. Categorías
2. Productos
3. Usuarios
4. Pedidos
0. Salir
Seleccione: 2

--- PRODUCTOS ---
1. Listar
2. Listar por categoría
3. Crear
4. Editar
5. Eliminar
0. Volver
Seleccione: 3

Categorías disponibles:
[1] Hamburguesas | Listado de hamburguesas
Nombre: Double Chesse
Descripción: Hamburguesa doble carne, 130grs con queso cheddar
Precio: 12000
Stock: 5
Imagen (ruta/nombre):
¿Disponible? (S/N): s
Id de categoría: 1
Producto creado con id: 1

--- PRODUCTOS ---
1. Listar
2. Listar por categoría
3. Crear
4. Editar
5. Eliminar
0. Volver
Seleccione: █
```

Alta de usuario y mensaje de error por mail duplicado.

```

=== SISTEMA DE PEDIDOS (FOOD STORE) ===
1. Categorías
2. Productos
3. Usuarios
4. Pedidos
0. Salir
Seleccione: 3

--- USUARIOS ---
1. Listar
2. Crear
3. Editar
4. Eliminar
0. Volver
Seleccione: 2
Nombre: Alejandro
Apellido: Maure
Mail: alejandrodanielmaure@gmail.com
Celular: 1122334455
Contraseña: 123456
Roles: 1-ADMIN 2-USUARIO
Seleccione rol: 1
Usuario creado con id: 1

--- USUARIOS ---
1. Listar
2. Crear
3. Editar
4. Eliminar
0. Volver
Seleccione: 2
Nombre: Alejandro
Apellido: Maure
Mail: alejandrodanielmaure@gmail.com
Celular: 1122334455
Contraseña: 123456
Roles: 1-ADMIN 2-USUARIO
Seleccione rol: 1
Error: Ya existe un usuario con ese mail.

--- USUARIOS ---
1. Listar
2. Crear
3. Editar
4. Eliminar
0. Volver
Seleccione: 1

Usuarios activos:
[1] Alejandro Maure | alejandrodanielmaure@gmail.com | 1122334455 | rol: ADMIN

```

Creación de pedido con detalles y total calculado.

```

--- PEDIDOS ---
1. Listar
2. Listar por usuario
3. Crear
4. Actualizar estado / forma de pago
5. Eliminar
0. Volver
Seleccione: 3

Usuarios disponibles:
[1] Alejandro Maure | alejandrodanielmaure@gmail.com | 1122334455 | rol: ADMIN
[2] Dani Saraza | danisaraza@gmail.com | 1133225544 | rol: USUARIO
Id de usuario: 2
Estados: 1-PENDIENTE 2-CONFIRMADO 3-TERMINADO 4-CANCELADO
Seleccione estado: 1
Formas de pago: 1-TARJETA 2-TRANSFERENCIA 3-EFECTIVO
Seleccione forma de pago: 1

Productos disponibles:
[1] Double Chesse | $12000,00 | stock: 5 | cat: Hamburguesas | disp: Sí
[2] Honey | $5200,00 | stock: 20 | cat: Cervezas | disp: Sí
Id de producto: 2
Cantidad: 2
¿Agregar otro detalle? (S/N): S

Productos disponibles:
[1] Double Chesse | $12000,00 | stock: 5 | cat: Hamburguesas | disp: Sí
[2] Honey | $5200,00 | stock: 20 | cat: Cervezas | disp: Sí
Id de producto: 1
Cantidad: 1
¿Agregar otro detalle? (S/N): n
¿Confirmar creación del pedido? (S/N): s
Pedido creado con id: 1 | Total: $22400.0

--- PEDIDOS ---
1. Listar
2. Listar por usuario
3. Crear
4. Actualizar estado / forma de pago
5. Eliminar
0. Volver
Seleccione: 1

Pedidos:
[1] 2026-06-16 | Dani Saraza (#2) | PENDIENTE | $22400,00 | TARJETA
- Detalle #1: Honey x 2 => $10400,00
- Detalle #2: Double Chesse x 1 => $12000,00

```

Error de stock insuficiente al crear pedido.

```

--- PEDIDOS ---
1. Listar
2. Listar por usuario
3. Crear
4. Actualizar estado / forma de pago
5. Eliminar
0. Volver
Seleccione: 3

Usuarios disponibles:
[1] Alejandro Maure | alejandrodanielmaure@gmail.com | 1122334455 | rol: ADMIN
[2] Dani Saraza | danisaraza@gmail.com | 1133225544 | rol: USUARIO
Id de usuario: 2
Estados: 1-PENDIENTE 2-CONFIRMADO 3-TERMINADO 4-CANCELADO
Seleccione estado: 1
Formas de pago: 1-TARJETA 2-TRANSFERENCIA 3-EFECTIVO
Seleccione forma de pago: 2

Productos disponibles:
[1] Double Chesse | $12000,00 | stock: 4 | cat: Hamburguesas | disp: Sí
[2] Honey | $5200,00 | stock: 18 | cat: Cervezas | disp: Sí
Id de producto: 1
Cantidad: 20
¿Agregar otro detalle? (S/N): n
¿Confirmar creación del pedido? (S/N): s
Error: Stock insuficiente para Double Chesse. Disponible: 4
El pedido no fue registrado.

```

Eliminación lógica con confirmación S/N.

```

--- PEDIDOS ---
1. Listar
2. Listar por usuario
3. Crear
4. Actualizar estado / forma de pago
5. Eliminar
0. Volver
Seleccione: 5

Pedidos:
[1] 2026-06-16 | Dani Saraza (#2) | PENDIENTE | $22400,00 | TARJETA
  - Detalle #1: Honey x 2 => $10400,00
  - Detalle #2: Double Chesse x 1 => $12000,00
Id de pedido a eliminar: 1
¿Confirma eliminación? (S/N): s
Pedido eliminado lógicamente.

```

4. DIFICULTADES Y SOLUCIONES

4.1 Pedido inconsistente ante error de stock

Problema: Si se guarda el pedido en el repositorio antes de validar todos los detalles, un fallo a mitad de proceso deja datos huérfanos.

Solución: PedidoService construye el pedido completo en memoria local. Solo llama a pedidoRepository.guardar() cuando addDetallePedido, calcularTotal y descontarStockProductos finalizan sin excepción.

4.2 Mismo producto agregado dos veces en un pedido

Problema: Stock podría validarse por línea sin considerar cantidad acumulada.

Solución: Método privado cantidadYaPedida() en Pedido suma cantidades del mismo producto ya cargadas en el pedido antes de validar stock disponible.

4.3 Entrada inválida en consola

Problema: Scanner.nextInt() deja salto de línea y provoca InputMismatchException.

Solución: ConsolaUtil lee siempre con nextLine() + parse, en bucle hasta obtener valor válido.

4.4 Integridad al eliminar categoría

Problema: La consigna permite interpretar si se puede borrar categoría con productos asociados.

Solución: Categoria.tieneProductosActivos() + validación en CategoriaService eliminar(). Se impide la baja y se informa al usuario.

4.5 Trabajo en paralelo sin conflictos masivos

Problema: Dos desarrolladores en el mismo repositorio generan conflictos.

Solución: División por épicas, cookbook previo, archivos compartidos mínimos (AppContext, MenuPrincipal) y merges secuenciales por PR revisado.

5. HISTORIAS DE USUARIO Y CÓDIGO

ÉPICA 1 – CATEGORÍAS ()

HU-CAT-01 MenuCategoria.listar() + CategoriaService.listarActivas()
HU-CAT-02 MenuCategoria.crear() + CategoriaService.crear()
HU-CAT-03 MenuCategoria.editar() + CategoriaService.editar()
HU-CAT-04 MenuCategoria.eliminar() + CategoriaService.eliminar()

ÉPICA 2 – PRODUCTOS

HU-PROD-01 MenuProducto.listarGeneral() / listarPorCategoria()
HU-PROD-02 MenuProducto.crear() + ProductoService.crear()
HU-PROD-03 MenuProducto.editar() + ProductoService.editar()
HU-PROD-04 MenuProducto.eliminar() + ProductoService.eliminar()

ÉPICA 3 – USUARIOS

HU-USR-01 MenuUsuario.listar()
HU-USR-02 MenuUsuario.crear() + UsuarioService.crear()
HU-USR-03 MenuUsuario.editar() + UsuarioService.editar()
HU-USR-04 MenuUsuario.eliminar() + UsuarioService.eliminar()

ÉPICA 4 – PEDIDOS

HU-PED-01 MenuPedido.listarGeneral() / listarPorUsuario()
HU-PED-02 MenuPedido.crear() + PedidoService.crearPedidoConDetalles()
+ Pedido.addDetallePedido() + Calculable.calcularTotal()
HU-PED-03 MenuPedido.actualizar() + PedidoService.actualizarEstadoYPago()
HU-PED-04 MenuPedido.eliminar() + PedidoService.eliminar()

6. BIBLIOGRAFÍA Y WEBGRAFÍA

- Oracle Corporation. The Java™ Tutorials.
<https://docs.oracle.com/javase/tutorial/>
- Oracle Corporation. Java SE 21 Documentation.
<https://docs.oracle.com/en/java/javase/21/>
- Oracle Corporation. Collections Framework Overview.
<https://docs.oracle.com/javase/tutorial/collections/>
- Oracle Corporation. Exceptions (The Java™ Tutorials).
<https://docs.oracle.com/javase/tutorial/essential/exceptions/>
- Oracle Corporation. Enum Types (The Java™ Tutorials).
<https://docs.oracle.com/javase/tutorial/java/javaOO/enum.html>
- Oracle Corporation. LocalDate / LocalDateTime (java.time).

<https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/time/package-summary.html>

- Universidad Tecnológica Nacional. Consigna TPI Programación 2 – Food Store (documento oficial de la cátedra, 2026).